



IBM BOB 2.0 HACKATHON · SEP 27, 2026

NodeFlow

We built NodeFlow from **written specifications**: every request became a versioned contract, **IBM Bob 2.0** implemented it, and an independent review audited the delivery against that same text.

Tauri · Rust · React Flow · TypeScript · public repo · web demo, nothing to install

An unclear request creates extra work in implementation, in review and in fixes

What it costs today

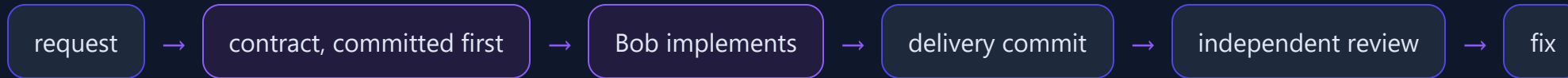
- **Implementation:** you build too much or too little, because the scope was never written down.
- **Review:** there is nothing to compare against, so review turns into opinion.
- **Fixes:** the defect shows up late, when other code already sits on top of it.

What we did instead

Every request starts as a **contract committed before any code**. The agent implements against that contract, and an **independent review** audits the delivery against the same text.

This is how we built NodeFlow itself — and it is the workflow we are submitting: **the same chain works for any request**, and the contract templates ship in the repository.

From a request to reviewed code, with evidence at every step



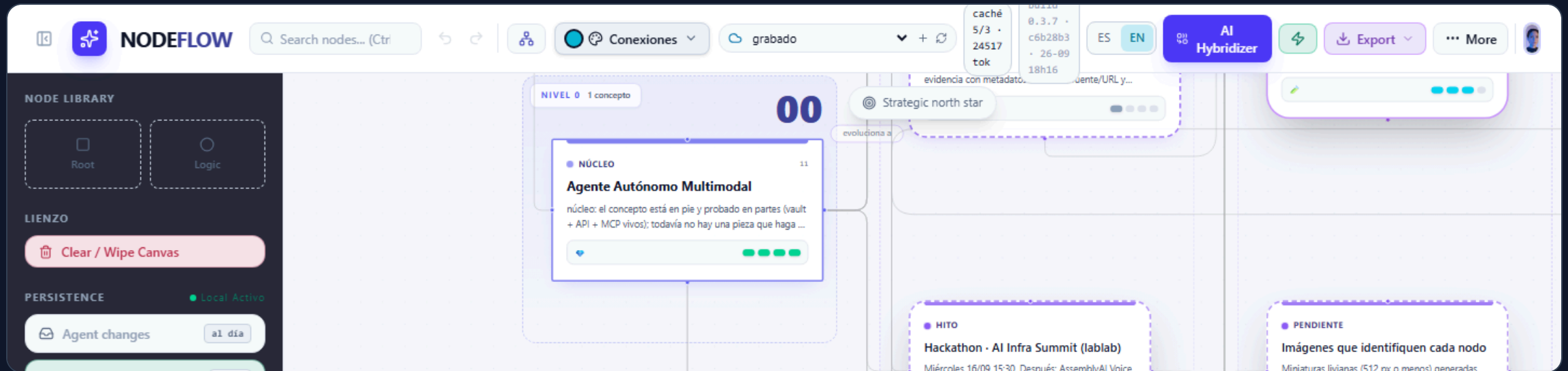
Two real requests, with their commits:

- **The partial-transcript segmenter.** The contract was committed before any code (`docs/hackathon/PEDIDO-BOB-01.md`). Bob implemented the module and its endpoint and delivered `cff0baa`. A later revision fixed its stability: `e84fb5b`.
- **Creating the nodes when the turn closes.** The integration landed in `8582026`; the review found that nodes were being created once per audio segment instead of once per turn, and that was fixed in `0e059ed`.

The steps are **manual and deliberate**: this is not an automatic voice-to-code pipeline, and the review is a different agent from the one that wrote the code.

A canvas that draws itself while you speak

A desktop app (Tauri + Rust + React Flow): a **deterministic segmenter** on the backend decides whether what you said is an idea, a correction or noise; a **draft** appears as you speak; when the turn closes, each topic becomes a **real node** —chained, draggable and connectable—. Nodes are created **without a confirmation step**: the safety net is that you can undo them. The interface ships in **Spanish and English** (the ES/EN switch is in the top bar).



The canvas in the **web demo** at nodeflowsss.netlify.app, where dictation is **real**: the server issues a temporary speech session, so anyone can speak without an account or a key. The video shows the desktop app creating these nodes by dictation.

Bob implemented the segmenter and its client. The chain proves it.

What Bob implemented

- The backend **partial-transcript segmenter** (Rust) and its endpoint.
- The **partial-result client** on the frontend.
- The **UX fixes** that came out of the first real-audio test.

Where the evidence lives

- The **contracts**, committed before the code:
`docs/hackathon/PEDIDO-0*.md`
- Bob's session record — it logs the `read_file` on the request:
`bob_sessions/bob-task-*.md`
- The **task session summaries** and which files it touched:
`bob_sessions/ · docs/hackathon/EVIDENCIA-BOB.md`

What we do **not** claim: no parallelism and no subagents attributed to Bob, and no credit for the transcription or speech engines — those are hired services.

What the repository records

324_{/324}

Rust tests
commit 0d05b2f

141_{/141}

frontend tests
commit e47537a

5

verification levels
docs/SISTEMA-ORQUESTACION.md

1

defect caught in review
before merging · 0e059ed

What these numbers are

They are **recorded results from the test suites and the pull-request history** — what was verified at the time, on the commits named next to each one, published with the repository so they can be checked. They are **not** a measured time saving.

The limits, stated up front

- Test counts measure **verification**, not saved effort.
- The defect caught in review is **one case**, not a controlled experiment.
- The process was applied **to this project**: it does not prove general productivity.

Any developer can run the same chain

The five steps

- **Write the contract** — scope, signature, enumerated cases. Templates are in the repo ([docs/hackathon/PEDIDO-0*.md](#)).
- **Commit it before the code**, so the request has a fixed date.
- **Let the agent implement against that text**, not against a chat.
- **Audit the delivery with a different agent** — whoever wrote the code does not grade it.
- **Fix and record it**: the fix commit points back at the review finding.

Why it held

The contract is the thing you can compare against, so review stops being opinion. When the contract left something open — the cardinality of an event — the review found it and the fix landed before merge.

The canvas is where the request becomes that contract: you talk, the nodes appear, and it gets written down while it is still fresh.

The process, reusable. The product, working.

Try it

Repo · github.com/NeonEden/nodeflow-ibm-bob

Web demo · nodeflowsss.netlify.app (real dictation: the server issues a temporary speech session — 30 s per session, 3 per day)

Desktop app · a Windows installer published in Releases, or build it with `npx tauri dev`

What we bring

A workflow where the request is written before the code, the agent implements against that text and the delivery is audited by a different agent — with **IBM Bob 2.0 implementing the segmenter and its client**, and the evidence of every step in the repository.

Presented by Tomás · Sep 27, 2026 · IBM Bob 2.0 Hackathon